My JS Syllabus

Phase 1: Foundations & Memory (Sections 1, 2, 3)

Section 1: JavaScript Foundations (Absolute Beginner)

1.1 JavaScript Basics & Environment

- What is JavaScript
- JavaScript vs Java vs Other Languages
- ECMAScript Standard
- ES Versions (ES3 → ES2024+)
  - Introduction to Modern Features (ES6, ES7) – Detailed in ES6+ Features Section
- JS Engines (V8, SpiderMonkey, JavaScriptCore)
- Browser JS vs Node.js JS
- How JS Runs in Browser
- Script Loading - `script`, `defer`, `async`
- Introduction to JavaScript, Why It Is Important, and What Can It Do for You?
- How to Link JavaScript Files Using Script-Tag
- Running JavaScript in the Browser Console

1.2 Syntax & Language Rules

- Statements vs Expressions
- Semicolons (ASI – Automatic Semicolon Insertion) and Semicolon rules.
- Identifiers: Naming rules, Case Sensitivity, Reserved Words.
- Keywords & Reserved Words
- Comments - Single-line, Multi-line, and JSDoc intro.
- Logging with JavaScript
  - Logging API: `console.log`, `info`, `warn`, `error`, `table`, `trace`.
- Interactions: `alert()`, `prompt()`, `confirm()`.

1.3 Variables (Very Important)

- `var` vs `let` vs `const`.
- The Lifecycle: Declaration, Initialization, Assignment.
- Re-Declaration & Re-Assignment Rules

- Global Scope Pollution
- Best Practices for Variables
- Scope (Global, Block, Functional)
- Hoisting Impact per Type - Variable hoisting vs. Function hoisting.
- Temporal Dead Zone (TDZ)
- Q1. Why var Leaks Out of a Block {} but let Doesn't
- Q2. Why const Allows Updation in Property Values of an Object

1.4 Data Types (Complete)

Primitive Types - `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`.

Non-Primitive - `object`, `array`, `function`, `date`, `regexp`, `map`, `set`.

Special Values: `NaN` (Not a Number), `undefined`, `null`, `Infinity`, `-0`.

Concepts

- Primitive vs Reference
- Memory: Stack (Primitives) vs. Heap (Reference types), Pass by Value vs. Pass by Reference.
- The Quirks: `typeof null === 'object'`, `typeof NaN === 'number'`.
- Type Coercion / Dynamic Typing - Explicit Conversion (Type Casting), Implicit Conversion (Type Coercion), Truthy vs. Falsy values, `isEmpty()` logic and checking for existence.
- == vs ===
- Q2. Why "5"+1 `= '51' Whereas "5" - 1 =` 4

Section 2: Operators (Full Coverage)

2.1 Operators

Arithmetic

-
  -
    -
      - / % **

Unary Plus/Minus - ++, --, unary +, unary - (Increment / Decrement pre/post ++, --)

Comparison

- `==`, `!=`, `===`, `!==`, `<`, `>`, `<=`, `>=`
- Object Comparison

Logical

- `&&` (AND), `||` (OR), `!` (NOT)
- Short-Circuiting evaluation
- Nullish Coalescing `??`
- !! Operator

Assignment

- `=`, `+=`, `-=`, etc., and Logical Assignment (`||=`, `&&=`, `??=`

Bitwise: `&`, `|`, `^`, `~`, `<<`, `>>`, `>>>`

Special

- typeof, instanceof, in, delete, Optional Chaining `?.` , Spread `...` , Rest `...`
- Nested Conditions and Ternary Operator `? :`.
- !! Operator for Truthy/Falsy Value of Any Expression
- +"5" => 5
- Basic Operators (Arithmetic, Assignment, Increment, Decrement, Comparison, Logical, Bitwise) [+ , - , * , / , ++ , -- , == , === , != , and more]

Section 3: Numbers & Math

3.1 Numbers

- Integer vs Floating Point
- IEEE 754 Issues
- NaN
- Infinity

Number Methods

- Number()
- parseInt()
- parseFloat()
- isNaN()
- isFinite()

- toFixed()
- toPrecision()
- toString(radix)

3.2 Math Object (All)

- Math.abs
- Math.round
- Math.floor
- Math.ceil
- Math.trunc
- Math.random
- Math.min
- Math.max
- Math.pow
- Math.sqrt
- Math.cbrt
- Math.sign
- Math.clz32
- Math.imul

Modern Date & Time Handling + Intl API ★VERY IMPORTANT★

- Legacy Date object – common pitfalls & limitations
  - 0-based months
  - Daylight saving time issues
  - Mutability problems
- Creating dates
  - new Date() various constructors
  - Date.now()
  - Date.parse()
- Date getters & setters
  - getFullYear(), setHours(), etc.
- Modern & recommended approaches
  - Intl.DateTimeFormat – powerful formatting
    - Basic usage
    - Locale-specific formatting (en-US, hi-IN, fr-FR, etc.)

        - Options: dateStyle, timeStyle, weekday, era, year, month, day, hour...
    - Intl.RelativeTimeFormat – "2 hours ago", "in 3 days"
    - Intl.PluralRules – handling plurals ("1 item" vs "5 items")
    - Intl.NumberFormat – currency, percentages, compact notation
    - Intl.ListFormat – "apples, bananas, and oranges"
    - Intl.Collator – locale-aware string comparison/sorting
- Temporal Proposal status (as of 2026)
    - Very brief overview
    - Why it might replace Date in future
    - Current polyfill availability
- Popular libraries (brief mention)
    - date-fns (modern & tree-shakeable)
    - Luxon (powerful & timezone-friendly)
- Real-world examples
    - Formatting dates for different countries
    - Countdown timers
    - Age calculation
    - Event scheduling across timezones

🟡 Phase 2: Logic & Data Structures (Sections 4, 5, 6, 7)

Section 4: Strings (Every Method That Exists)

4.1 Strings – Core + Real World

Creation

- String Literals vs `String()` constructor vs Template Literals

Properties

- length

Character Access

- charAt()
- charCodeAt()

- at()
- Bracket Notation

Searching & Checking

- includes()
- startsWith()
- endsWith()
- indexOf()
- lastIndexOf()
- search()
- match()
- matchAll()

Manipulation

- slice()
- substring()
- substr() (Deprecated but Seen)
- replace()
- replaceAll()
- concat()
- repeat()
- padStart()
- padEnd()

Formatting

- toUpperCase()
- toLowerCase()
- trim()
- trimStart()
- trimEnd()
- normalize()

Conversion

- split()
- toString()

- valueOf()
- Working with Strings in JS and There [splice, slice, template string, split, replace, includes]

4.2 Regular Expressions – DEEP COVERAGE

- What are Regular Expressions and why they matter
- Creating RegExp
  - Literal notation /pattern/
  - RegExp constructor new RegExp()
- Flags (g, i, m, s, u, y, d, v)
- Important modern flag: /v flag (ES2024) – set notation & improved properties
- Core methods on String
  - match()
  - matchAll()
  - search()
  - replace()
  - replaceAll()
  - split()
- RegExp methods
  - test()
  - exec()
- Groups & Capturing
  - Named capture groups (?...)
  - Non-capturing groups (?:...)
- Lookahead & Lookbehind
  - Positive lookahead (?=...)
  - Negative lookahead (?!...)
  - Positive lookbehind (?<=...)
  - Negative lookbehind (?<!...)
- Common real-world patterns
  - Email validation
  - URL parsing
  - Password strength
  - Phone numbers (international)
  - Date formats
  - Currency extraction
- Performance considerations & common pitfalls

Section 5: Control Flow

5.1 Conditionals

- if
- if-else
- if-else if-else
- else
- Nested Conditions
- Ternary Operator
- switch
- break
- Fall-Through Behavior
- Understanding Condition Operator in JavaScript [if, else, if-else, else-if, Ternary Operator, switch]

5.2 Loops

- for
- while
- do...while
- forEach
- for...in
- for...of
- Recursion in JS
- break
- continue
- Labeled Loops
- for Loop in JavaScript
- while Loop in JavaScript
- do...while in JavaScript
- forEach in JavaScript
- for in Loop in JavaScript
- for of Loop in JavaScript
- Recursion in JavaScript
- Loop Control Statements [break, continue]
- Q1. for-in vs for-of

Section 6: Arrays (All Methods Used in Real Dev)

6.1 Arrays – Complete

Creation

- []
- Array()
- Array.of()
- Array.from()

Mutating Methods

- push
- pop
- shift
- unshift
- splice
- sort
- reverse
- fill
- copyWithin

Non-Mutating

- slice
- concat
- toSorted
- toReversed
- toSpliced
- with

Iteration

- forEach
- map
- filter
- reduce
- reduceRight

- some
- every
- find
- findIndex
- findLast
- findLastIndex

Searching

- includes
- indexOf
- lastIndexOf
- at
- join
- flat
- flatMap
- What Are Arrays in JavaScript and How to Create an Array
- Understand How to Accessing Elements in Array
- Functions on Arrays [push, pop, shift, unshift, indexOf, array destructuring, filter, some, map, reduce, spread operator, slice, reverse, sort, join, toString]
- Iterating Over Arrays Using [For Loop, forEach]
- Creating Arrays, Modify Arrays
- Array Methods: push, pop, shift, unshift, splice, slice, reverse, sort
- map, filter, reduce, forEach(), find, some, every
- Destructuring, Spread Operator
- Q1. splice vs slice
- Q2. map vs forEach
- Q3. sort Behaviour Without compareFn

Set Methods (ES2024+) ★NEW★

- Understanding Set as collection of unique values
- New Set methods (widely supported in 2025–2026)
  - union(other)
  - intersection(other)
  - difference(other)
  - symmetricDifference(other)
  - isSubsetOf(other)

    - isSupersetOf(other)
    - isDisjointFrom(other)
- Real-world use cases
    - Removing duplicates elegantly
    - Set operations on tags, permissions, user roles

Section 7: Objects (Real World Core)

7.1 Objects

- Understanding What Are Objects in JavaScript [Key-Value Pair]
- Object Literals
- Property Access
- Creating Objects, Accessing Properties, Deleting Property and Nested Objects
- Computed Properties
- Methods
- Shorthand Syntax
- Destructuring
- this Keyword

Object Methods

- Object.keys
- Object.values
- Object.entries
- Object.assign
- Object.freeze
- Object.seal
- Object.preventExtensions
- Object.hasOwn
- Object.getPrototypeOf
- Object.setPrototypeOf

this

- Global this
- Function this
- Arrow Function this
- Method this

- call
- apply
- bind
- Key Value Structure
- Dot vs Bracket [] Notation
- Nesting and Deep Access
- Object Destructuring
- Looping: for-in, Object.keys, Object.values, Object.entries
- Copying Objects: Spread, Object.assign, Deep Clone
- Optional Chaining ?.
- Computed Properties
- Delete Property
- Q1. Reference Copy vs Deep Clone (JSON.stringify, JSON.parse)
- Creating Property Key Using Variable, Using [varName]
- Destructuring Object and Giving Custom Name to Key {"first-name": firstName}
- Recognise How Objects Are Stored, Traverse Keys of an Object, Array as Object
- Operation in Objects [freeze, seal, destructuring, object methods, this keyword]

Phase 3: The Functional Core (Sections 8, 9)

Section 8: Functions (Deep)

8.1 Functions

- What Are Functions, Why They Are Used?
- Function Declaration
- Function Expression
- Arrow Functions
- Parameters vs Arguments
- Parameters in JS [Required, Destructured, Rest, Default]
- Arguments in JS [Positional, Default, Spread]
- Return Behavior
- Function Hoisting Difference in Function Declaration and Function Statement
- Classic Function, Nested Function (Function Within Function), Scope Chain in JavaScript
- Understanding Immediately Invoked Function Expression (IIFE)
- More Functions in JavaScript [Arrow Function, Fat Arrow, Anonymous, Higher Order, Callback, First Class, Pure Function, Impure Function, Recursion, Currying]

- Understanding Scoping in JS [Global Scope, Function Scope]
- Understanding Closures, Scoping Rule
- Understanding Function in JavaScript and Why It's Widely Used [Parameters, Arguments, Rest Parameters, Hoisting, Variable Hoisting, Function Hoisting]
- First Class Functions (Functions Can Be Used as Values)
- Higher Order Function (HOF)
- Pure vs Impure Functions
- Closures and Lexical Scoping, Private Variables, Environment `[[Scope]]`
- IIFE (Immediately Invoked Function Expression)
- Hoisting Differences Between Function Declarations and Function Expressions
- Arrow vs Regular Functions: this Context in Objects
- **The `this` Keyword:** Global, Method, Arrow, Constructor, and Explicit (`call`, `apply`, `bind`).
- Function Hoisting and TDZ
- Scope Chain and Closure Traps
- Q1. Create a Reusable Discount Calculator
- Q2. Build a Counter Using Closures
- Q3. Use IIFE to Isolate Variables
- Arrow Functions Don't Have Their Own this, They Take It from Their Parent
- Introduction to Closures – Detailed in Advanced Core JS

8.2 Execution Context

- Global Execution Context
- Function Execution Context
- Call Stack
- Stack Overflow
- Execution Context, Memory Creation & Execution Phase
  - Definition: A Hypothetical "Box" or Stage Created by JS for Functions to Run
  - Two Phases
    - Memory Phase: Allocates Space for All Variables and Functions Within the Execution Context
    - Execution Phase: Where the Actual Code Runs
  - Abstract Concept: Explained as Something That Exists but Cannot Be Physically Seen
- The Call Stack & Stack Overflow.
- The Heap & Garbage Collection (Mark-and-Sweep).

Section 9: Advanced Core JS

9.1 Scope & Closures

- Lexical Scope
- Closure Mechanics
- Private Variables
- Memory Implications
- Scope
  - Definition: Where Created Variables and Functions Can Be Used
  - Functional Scope: Variables Only Accessible Within Their Parent Function
  - Global Scope: Variables Accessible Throughout the Entire Code
  - Block Scope: Variables Accessible Only Within Curly Braces (e.g., if or while Blocks, Not Functions)
- Lexical Scoping vs. Dynamic Scoping
  - Lexical Scoping: JavaScript Follows This. It Depends on Where the Element Is Physically Written in the Code (Its Physical Location) to Determine What It Can Access
  - Dynamic Scoping: Not Followed in JavaScript. It Would Depend on Where the Function Is Called from to Determine Variable Access
- Closures: Definition & How Variables Are Preserved
  - Definition: Functions That Are Inside a Parent Function, Returned by the Parent Function, and Use a Variable from the Parent Function
  - Benefits
    - Can Make Variables Private
    - Prevents Global Pollution
  - Variable Preservation (Backlink): When a Closure Is Formed, JavaScript Creates a "Backlink" (or Copy) of the Function and Its Variables in an "Environment" (Represented as `[[Environment]]`). This Allows the Returned Function to Access the Parent's Variables Even After the Parent Function Has Finished Executing
  - Use Cases
    - Private Counters: Demonstrates How a Closure Can Maintain a Private Counter That Persists Its Value Across Multiple Calls
    - Encapsulation: Hiding the Inner Logic and Only Providing Access to Necessary Functionalities

9.2 Prototypes

- Prototype Chain
- proto
- prototype

- Inheritance Model

9.3 Classes (ES6)

- Class Syntax
- Constructor
- Methods
- Static Methods
- Getters & Setters
- Inheritance
- super

🔵 Phase 4: DOM & Browser APIs (Sections 10, 11, 12, 13, 14, 15)

Section 10: Browser & Web APIs

10.1 DOM

- Introduction to DOM in JavaScript
- document - DOM Structure and Tree [Nodes, Elements, Document], Nodes vs Elements, Live vs Static NodeLists.
- DOM Tree Structure - Node, Element, Text, Comment
- Selectors - [document.getElementById, document.getElementsByTagName, document.getElementsByClassName, `getElementById`, `querySelector`, `querySelectorAll`, `closest()`
- DOM Tree Traversal - parentNode, `parentElement`, childNodes, firstChild, `children`, `nextElementSibling`, `contains()`
- Manipulating DOM Element in JavaScript [innerHTML, textContent, setAttribute, getAttribute, style property, classList]
- Text/Content Access: innerText, textContent, innerHTML (Diff Between Each of These)
- Classes & Styles - `style` property, `getComputedStyle`, `classList` (add, remove, toggle)
- Style Updates Using .style or classList (add, remove, toggle)
- Attribute Manipulator: getAttribute, setAttribute, removeAttribute
- Create / Remove Elements - `createElement`, appendChild, insertBefore, removeChild, `append`, `prepend`, `insertAdjacentHTML`, `remove()`, `DocumentFragment`

10.2 Events

- Event Types
- Event Object
- Bubbling
- Capturing
- Delegation
- preventDefault
- stopPropagation
- Event Handling in JavaScript [addEventListener(), event bubbling, event.target]
- `addEventListener`: options (`once`, `passive`, `capture`), `removeEventListener`
- Event Delegation and `event.target` vs `event.currentTarget`
- Understanding Scroll Events, Mouse Events, Key Events and Strict Mode
- Browser Events [DOMContentLoaded, load, resize, scroll]
- Events And Event Handling
- Event Binding - addEventListener, removeEventListener
- Common Events - click, input, change, submit, mouseover, keyup, keydown, keypress
- Event Object - target, type, preventDefault
- Event Bubbling and Capturing
- Event Delegation
- event.target vs event.currentTarget
- Capturing Phases vs Bubbling Phases

10.3 Storage

- localStorage
- sessionStorage
- Cookies
- JSON Serialization
- Working with Storage [Local Storage, Session Storage, Cookies]
- localStorage, sessionStorage, And Cookies
- Local Storage API - setItem, getItem, removeItem, clear
- sessionStorage API
- Storing Retrieving Strings vs JSON
- Basic Cookie Structure Manual key=value; path=/ Format
- Why Only Strings Work in localStorage
- Cookies Need Manual Encoding/Expiration Handling
- `IndexedDB` (Intro)

10.4 Fetch & Networking

- fetch - API: Request/Response objects, Headers, Status codes.
- HTTP Methods
- Headers
- Status Codes
- CORS
- JSON Handling - `JSON.stringify()`, `JSON.parse()`
- Web APIs in DOM [Fetch API]
- `AbortController` (Canceling requests)

Section 11: DOM (Document Object Model) — Complete & Advanced

11.1 DOM Fundamentals

- What Is DOM
- DOM Tree Structure
- Nodes vs Elements
- Node Types
  - Element
  - Text
  - Comment
  - Document
- Live vs Static NodeLists
- DOM vs Virtual DOM (Conceptual)

11.2 DOM Selection (All Methods)

Basic Selectors

- getElementById
- getElementsByClassName
- getElementsByTagName

Modern Selectors

- querySelector
- querySelectorAll
- CSS Selectors Support

- Performance Differences

11.3 DOM Traversal (Full)

- parentNode
- parentElement
- children
- childNodes
- firstChild
- lastChild
- firstElementChild
- lastElementChild
- nextSibling
- previousSibling
- nextElementSibling
- previousElementSibling
- Closest Ancestor: closest()
- Checking Containment: contains()

11.4 DOM Manipulation (Core Real-World)

Create / Insert

- createElement
- createTextNode
- append
- appendChild
- prepend
- insertBefore
- insertAdjacentElement
- insertAdjacentHTML
- insertAdjacentText
- Document Fragments (DocumentFragment)

Remove / Replace

- remove
- removeChild
- replaceChild

- replaceWith

11.5 DOM Attributes & Properties

- getAttribute
- setAttribute
- removeAttribute
- hasAttribute
- Attribute vs Property Difference
- dataset
- Boolean Attributes
- ARIA Attributes

11.6 DOM Styling & Classes

- style Property
- Inline Styles vs Computed Styles
- getComputedStyle
- CSS Variables (--var)
- className
- classList
  - add
  - remove
  - toggle
  - contains
  - replace

11.7 DOM Content

- innerHTML
- outerHTML
- innerText
- textContent
- Security Implications (XSS)
- Performance Considerations

11.8 Forms & Inputs (Deep)

- Form Elements

- Input Types
- value
- checked
- selectedIndex
- name
- Form Submission Lifecycle
- FormData
- Client-Side Validation
- Constraint Validation API
- Custom Validation Messages
- Working with Forms and Input Elements [Accessing Form Data, Validating Forms, preventDefault(), onsubmit, onchange]
- Working with Classes: Adding, Removing, Toggling (classList Methods)
- Forms And Form Validation
- Reading Values from Input, Textarea, Select
- Prevent Default Submission
- Inline vs JS Based Validation
- Showing Error Message Conditionally
- Pattern Attribute vs Custom Regex
- Diff Between Value and textContent
- Form Submission vs Button Click

Section 12: Events System (Advanced)

12.1 Events Core

- Event Object
- Event Phases
  - Capturing
  - Target
  - Bubbling
- Default Behavior
- Cancelable Events

12.2 Event Handling

- addEventListener
- removeEventListener

- Anonymous vs Named Handlers
- Passive Listeners
- Once Option
- AbortController for Events

12.3 Event Types (Complete)

Mouse

- click
- dblclick
- mousedown
- mouseup
- mousemove
- mouseenter
- mouseleave
- mouseover
- mouseout
- contextmenu

Keyboard

- keydown
- keyup
- keypress (Deprecated)
- Key Codes vs Key Values

Form

- submit
- change
- input
- focus
- blur
- reset

Window / Document

- load

- DOMContentLoaded
- resize
- scroll
- visibilitychange

12.4 Event Delegation (Real World)

- Bubbling Based Delegation
- Dynamic Elements Handling
- Performance Benefits
- Common Pitfalls

Section 13: BOM (Browser Object Model)

13.1 Window Object

- Global Object
- window.open
- window.close
- window.moveTo
- window.resizeTo
- alert
- prompt
- confirm
- devicePixelRatio
- Browser Object Model [window, navigator, history, location, document]
- Window Object [window.location, window.history]

13.2 Location Object

- location.href
- location.protocol
- location.host
- location.pathname
- location.search
- location.hash
- assign
- replace
- reload

13.3 History API

- history.back
- history.forward
- history.go
- pushState
- replaceState
- SPA Navigation Concepts

13.4 Navigator Object

- navigator.userAgent
- navigator.platform
- navigator.language
- navigator.onLine
- navigator.geolocation
- Feature Detection

Section 14: Timers & Scheduling

14.1 Timers

- setTimeout
- clearTimeout
- setInterval
- clearInterval
- Recursive Timeouts
- Drift Issues
- Timers vs Event Loop
- Timing Events - setTimeout(), setInterval(), clearTimeout(), clearInterval()
- setInterval & setTimeout in JavaScript
- Timers And Intervals
- setTimeout, clearTimeout
- setInterval, clearInterval
- Delay Actions, Auto Refresh

14.2 Animation & Timing APIs

- requestAnimationFrame

- Frame Lifecycle
- Canceling Animations
- Performance Comparison with Intervals

Section 15: Web APIs (Complete Coverage)

15.1 Fetch API (Advanced)

- Request Object
- Response Object
- Headers API
- Streams
- AbortController
- Timeout Handling
- Retry Logic
- File Uploads
- Download Blobs
- Fetch API + HTTP Basics

15.2 Storage APIs

- localStorage
- sessionStorage
- Storage Events
- Quota Limits
- IndexedDB (Intro)
- Cache API

15.3 Media APIs

- Audio API
- Video API
- Media Events
- Media Controls
- Fullscreen API
- Picture-in-Picture

15.4 File & Clipboard APIs

- File Input Handling
- FileReader
- Blob
- URL.createObjectURL
- Clipboard API (read/write)

15.5 Observer APIs (Very Important)

- IntersectionObserver
- MutationObserver
- ResizeObserver
- Performance Use Cases

15.6 Geolocation API

- Permissions
- Accuracy
- Error Handling
- Real-World Use

15.7 Web Workers

- Dedicated Workers
- Shared Workers
- Worker Communication
- Offloading Heavy Tasks

🟣 Phase 5: Asynchronous JavaScript (Sections 16, 17, 18)

Section 16: Asynchronous JavaScript

16.1 Asynchronous JS

- Synchronous vs Asynchronous
- Call stack, web apis, Callbacks, callback queue
- Callback Hell
- **Task Queues:** Microtasks (Promises, `queueMicrotask`) vs. Macrotasks (setTimeout, Events)

- Starvation of the Macrotask queue
- Introduction to Asynchrony in JavaScript
- Introduction to Callbacks and Problems in Callbacks
- How to Prevent Callback Hell Using async & await

16.2 Promises

- Promise States - Pending, Fulfilled, Rejected
- Understanding Promises [Pending, Resolved, Rejected]
- Methods: `then()`, `catch()`, `finally()`
- Chaining
- Combinators: `Promise.all()`, `allSettled()`, `race()`, `any()`

16.3 async / await

- Error Handling
- Parallel Execution
- Sequential Execution

Section 17: Advanced & Internals

17.1 Event Loop

- Call Stack
- Web APIs
- Microtasks
- Macrotasks
- Rendering Cycle

17.2 Memory Management

- Heap vs Stack
- Garbage Collection
- Memory Leaks

17.3 Modules

- import
- export
- Named vs Default

- Dynamic Imports

Section 18: Advanced Async & Concurrency

18.1 Event Loop (Deep Dive)

- Rendering Pipeline
- Microtask Queue
- Macrotask Queue
- Priority Order
- queueMicrotask
- Starvation Problems

18.2 Async Patterns

- Sequential Async Flows
- Parallel Async Flows
- Race Conditions
- Cancellation Patterns
- Retry Strategies
- Backoff Algorithms

🔴 Phase 6: OOP & Design Patterns (Sections 19, 20, 21)

Section 19: OOP in JavaScript (Real World)

19.1 OOP Concepts

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism
- Introduction to OOPS in JavaScript
- Understanding Classes and Objects in JavaScript
- Understanding Constructor and Prototypes [this keyword, call, apply, bind]
- More Topics in OOPS [class expression, hoisting, inheritance, getter & setter]

19.2 OOP Patterns in JS

- Constructor Functions
- Prototype-Based OOP
- Class-Based OOP
- Factory Functions
- Module Pattern
- Revealing Module Pattern
- Mixins

19.3 SOLID Principles (JS Context)

- Single Responsibility
- Open/Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

Section 20: Object-Oriented JavaScript

- Prototypes & Prototype Chain.
- **Classes:** Constructors, `super`, Static methods, Private fields `#`, Getters/Setters.
- **Design Patterns:** Module Pattern, Factory Functions, Observer Pattern (YouTube example), Singleton.
- **SOLID Principles** in JavaScript.
- Introduction to OOPS in JavaScript
- Understanding `classes` and `objects` in JavaScript
- Understanding `Constructor` and `Prototypes` - [`this keyword`, `call`, `apply`, `bind`]
- More Topics in OOPS - [`class expression`, `hoisting`, `inheritence`, `getter & setter`]

Section 21: Design Patterns in JavaScript

- Understanding What "Design Patterns" Mean in Coding
- Importance of Design Patterns for Maintainable, Scalable, and Modular Code
- Overview of Patterns to Be Covered
  - Module Pattern (IIFE-Based Pattern)
  - Revolving Module Pattern
  - Factory Function Pattern

- Observer Pattern

⚫ Phase 7: Professional Engineering (Sections 22, 23, 24, 25, 26)

Section 22: Professional Topics

22.1 Functional Programming

- Immutability
- Pure Functions
- Currying
- Composition

22.2 Security

- XSS
- CSRF
- Sanitization

22.3 Performance

- Debouncing
- Throttling
- Lazy Loading
- Code Splitting

Section 23: Industry Level

23.1 Tooling

- npm
- Bundlers
- Linters
- Formatters

23.2 TypeScript (JS Superset)

- Types

- Interfaces
- Generics
- Compilation

Section 24: Browser Performance & Security

24.1 Performance

- Reflows & Repaints
- Layout Thrashing
- DOM Batching
- Virtualization
- Lazy Rendering

24.2 Security (Browser)

- XSS via DOM
- CSRF Tokens
- Content Security Policy
- Sanitization Libraries
- Same-Origin Policy

Section 25: Introduction to Real-World Engineering Practices

- Difference Between College-Level Code and Real-World Engineering Code
- Importance of Optimized Code Practices
- Thinking and Coding Like a True Engineer

Section 26: Kuch Baatein Advance JavaScript Pr ⚙️

- Throttling and Debouncing Uses in JavaScript
- JSON Handling and JavaScript [JSON.parse(), JSON.stringify()]